



National University
of computer and emerging sciences

CrossViewNet

Predicting Ground Level Scene Layout from Aerial Imagery

Report

Field	Detail
University	FAST NUCES, Karachi
Program	BS Computer Science
Course	Fundamentals of Computer Vision (FCV)
Team Members	Anas Ahmed (22K-4154), Hamdan Vohra (22K-4318), Zain Abbas (22K-4502)
Date	8 May 2026

1. Abstract

This report documents our work on CrossViewNet, a cross view deep learning framework that learns to predict the ground level semantic layout of a scene using only the aerial photograph of that location. The original framework was introduced by Zhai et al. at CVPR 2017 and trains a convolutional network to bridge the aerial and ground perspectives through a learned spatial transformation, removing the need for any manually annotated aerial data.

Our team carried out a complete PyTorch reimplementation of the model on the CVUSA dataset, ran the full ten epoch training schedule on a consumer grade laptop GPU, and extended the pipeline with a Pix2Pix conditional GAN that turns the predicted semantic map into a photorealistic ground panorama. We also performed a detailed critique of the original architecture and codebase, identified ten concrete limitations along with eight promising research gaps, and drafted three targeted improvements that we propose for future work, namely a ResNet50 backbone, a CBAM attention block inside the transformation network, and DeepLabV3+ pseudo labels in place of the original SegNet labels.

2. Introduction

Semantic understanding of aerial imagery has wide ranging applications in urban planning, autonomous navigation, mapping and geospatial intelligence. The bottleneck for most modern approaches is data. Pixel level annotated aerial datasets are expensive to produce and tend to be specific to a single city or sensor, which limits how well any supervised model trained on them can generalise.

CrossViewNet sidesteps this bottleneck in an elegant way. Co located ground level panoramas are freely available through services like Google Street View, and they share semantic content with the aerial image at the same GPS coordinate. A road visible from the satellite is also visible from the street, the same is true for a building or a tree. By training a network to predict the semantic layout of the ground panorama purely from the aerial image, the model is forced to learn a meaningful aerial representation without any aerial labels at all. The ground panorama itself is segmented by an off the shelf segmentation network, and that pseudo label becomes the supervision target.

This project was undertaken as part of the Fundamentals of Computer Vision course in Semester 8 at FAST NUCES, Karachi. Our objectives were the following.

- Understand and reproduce the CrossViewNet architecture from the original CVPR 2017 paper.
- Modernise the legacy TensorFlow 1 codebase into a clean PyTorch 2.x implementation.
- Identify limitations in the original design and propose feasible improvements.
- Extend the pipeline with a Pix2Pix GAN stage that synthesises a photorealistic ground level panorama from the predicted semantic map.

3. Task Definition

The end to end task is straightforward to state and surprisingly hard to solve. Given a single top down satellite tile of a location, predict what the street level scene at that location looks like. We split this into two tightly coupled subtasks.

3.1 Subtask 1, Cross View Semantic Label Transfer

Given a 224 by 224 satellite tile, the model produces a semantic segmentation of the corresponding ground level 360 degree panorama. Concretely, the network maps a 17 by 17 grid of aerial features to an 8 by 40 grid of ground level class scores. The two grids share no visible pixels in common, so the network must learn an abstract geometric transformation that says, for every ground location, which aerial pixels contribute to its semantic label.

3.2 Subtask 2, Semantic Map to RGB Synthesis

Given the predicted semantic map, the second stage synthesises a photorealistic RGB panorama of the same scene. This stage is a straightforward image to image translation problem and is solved with a Pix2Pix style conditional GAN. Together the two stages turn an aerial photo into a synthesised ground panorama, going aerial photo, then semantic label map, then synthesised RGB ground panorama.

The fundamental difficulty of the first stage is that aerial and ground views are geometrically completely different. From above you see rooftops and tree canopies. From the street you see facades and the horizon. There is essentially zero pixel level overlap, and a model that tried to match pixels directly would fail. The model must instead learn a soft geometric lookup that connects the two viewpoints.

4. Original Paper Summary

Zhai et al. propose a training strategy that uses pixel level semantic labels extracted from ground panoramas as weak supervision for aerial image understanding. The architecture has four functional modules and is trained end to end on roughly 35,000 image pairs from CVUSA.

4.1 Network Architecture

- Aerial Feature Extractor (A). VGG16 backbone with VALID padding, combined with a PixelNet style hypercolumn formed from blocks 1 to 4. The output is a 17 by 17 by C semantic feature map.
- Scene Feature Extractor (S). Extracts a per pixel conditioning scalar from the deepest conv4 features. This signal modulates the transformation matrix.
- Transformation Network (F). A small three layer 1 by 1 convolution MLP that predicts each element of the transformation matrix M , conditioned on normalised position coordinates and the conditioning scalar.
- Transformation Module (T). Applies M to map the flattened aerial feature map to the ground viewpoint, producing an 8 by 40 output. A learnable per row ground bias is added on top.

Training minimises soft cross entropy between the predicted ground logits and the SegNet generated pseudo ground truth that has been one hot encoded and bilinearly downsampled to the logit resolution.

4.2 Applications Demonstrated in the Paper

Task	Method	Dataset
Weakly supervised aerial segmentation	Direct CNN output	CVUSA
Aerial pre training	Fine tune on ISPRS after cross view pre training	ISPRS Vaihingen

Task	Method	Dataset
Orientation estimation	Cross entropy sliding window over orientations	CVUSA, Cityscapes
Ground view synthesis	Deep generative model (Kim and Bengio 2016)	CVUSA

4.3 Key Results

On the ISPRS aerial segmentation benchmark, cross view pre training outperforms both random initialisation and ImageNet pretrained VGG16 across most training set sizes, demonstrating strong domain transfer. The model achieves particularly high precision on the Building, Low Vegetation and Tree classes.

5. Dataset Description

We used a subset of the CVUSA (Cross View USA) dataset, referred to in our codebase as `cvpr_subset`. CVUSA was specifically designed for cross view learning. Every sample is a matched triple, a satellite image, a ground level 360 degree panorama, and a semantic label map, all from the exact same GPS coordinate. The satellite imagery comes from Bing Maps, the panoramas come from Google Street View, and the panoramas are aligned to face north so that there is a consistent geometric relationship between the two views that the network can exploit.

5.1 Scale

Split	Sample Count
Training set	35,532 paired samples
Validation set	8,884 paired samples
Total geo tagged location triplets	44,416

5.2 Modalities per Sample

Modality	Format	Details
Aerial image	RGB JPEG	Satellite tile, top down view, square crop centred on the GPS location, roughly 224 by 224 pixels
Ground panorama	RGB JPEG	Street level 360 degree equirectangular panorama, target resolution 224 by 1232

Modality	Format	Details
Semantic label map	Grayscale PNG	Per pixel class annotation of the ground panorama, four classes

5.3 Semantic Classes

Class ID	Semantic Label	Visualisation Colour
0	Sky	Red
1	Vegetation	Green
2	Building	Blue
3	Road	Yellow

The label maps are not human annotated. They were generated by running a pretrained SegNet segmentation model on the ground panoramas. This is weak supervision. The labels are cheap to produce at scale but they carry the noise of the labelling network into our training data, which is one of the most important limitations of the entire approach.

6. Data Preprocessing

Raw images cannot be fed straight into the network. The architecture requires fixed input sizes, training is more stable when inputs are in a consistent numerical range, and the labels need to be in the right format for the loss function. Each modality has its own pipeline.

6.1 Aerial Image Pipeline

- Force three channel RGB regardless of source format using `Image.open().convert('RGB')`.
- Centre crop to 224 by 224. Cropping is preferred over resize because resize introduces blur and changes spatial scale relationships in the image. If the source image is smaller than 224 by 224 we fall back to a bilinear resize.
- Normalise to the range from minus one to plus one using `torchvision.transforms.functional.to_tensor` followed by `normalise(mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5])`. This matches the convention expected by the VGG16 VALID padded backbone.

The 224 by 224 input size is not arbitrary. With VALID padding and four pooling stages the VGG16 backbone produces a 17 by 17 feature map, and the value 17 by 17 equals 289, which directly determines the size of the transformation weight matrix. Changing the input size would break the entire geometric construction.

6.2 Ground Panorama Pipeline

- Force three channel RGB.
- Bilinear resize to 224 by 1232 (height by width, aspect ratio about 1 to 5.5). The aspect ratio reflects the equirectangular panoramic format. We do not crop here, because cropping would discard part of the 360 degree sweep.
- Same normalisation to the range from minus one to plus one.

For GAN training the panorama is instead resized to 256 by 512, normalised by pixel divided by 127.5 minus 1. The 256 by 512 resolution gives clean power of two dimensions for the U Net and is the largest size that fits in 4 GB of VRAM at batch size two.

6.3 Semantic Label Pipeline

Labels need special treatment because they encode discrete class identifiers, not continuous values. Bilinear interpolation of an integer label map would produce meaningless intermediate values like 1.5, so we always use nearest neighbour interpolation when resizing labels.

For CrossNet training we force the label map to a single channel, resize it with nearest interpolation to 224 by 1232, and convert to a long tensor of class identifiers with shape (H, W). Then at loss time we one hot encode the label to (B, C, H, W) float tensor and bilinearly downsample that to the logit resolution of 8 by 40. This produces soft probability targets, where boundary pixels end up as fractional mixtures like [0.0, 0.3, 0.7, 0.0] for a pixel that is 30 percent class 1 and 70 percent class 2. Soft targets give the model a more honest objective at class boundaries than forcing a hard binary choice at every coarse cell.

For GAN training we one hot encode the label and map it to the range from minus one to plus one using $\text{one_hot} \times 2 - 1$, so that class present cells become plus one and class absent cells become minus one. This matches the Tanh output range of the generator.

6.4 Augmentation

No online augmentation was applied during training. We did not use random flips, colour jitter or rotation, which is consistent with the original paper. We flagged augmentation as a clear improvement opportunity in the research plan.

7. Network Architecture

The pipeline is split into two stages on purpose. A single network that tries to go straight from aerial pixels to a photorealistic ground panorama would need to solve both viewpoint transformation and image synthesis at the same time, with a very weak learning signal. By separating concerns we let CrossNet solve the cross view transfer problem against clean semantic supervision, and we let the GAN

solve image synthesis against clean (semantic map, photo) pairs. Each model gets a well defined subproblem.

7.1 Stage 1, CrossNet

7.1.1 Backbone, VGG16 with VALID Padding

Standard VGG16 uses SAME (zero padded) convolutions so that spatial size is preserved through 3 by 3 conv layers. CrossNet uses VALID padding instead, where each 3 by 3 conv trims one pixel off every edge of the feature map. This is required for a specific geometric reason. With VALID padding and four pooling stages, a 224 by 224 input produces exactly a 17 by 17 final feature map. The 17 by 17 equals 289 source pixels, and that number directly determines the size of the transformation weight matrix. SAME padding would give a 14 by 14 final map, which would break the entire setup.

The spatial sizes produced by the backbone are summarised below.

Layer	Output Size	Channels
Input	224 by 224	3
Block 1, 2x Conv 3x3 VALID	220 by 220	64
Pool 1, MaxPool 2x2	110 by 110	64
Block 2, 2x Conv 3x3 VALID	106 by 106	128
Pool 2, MaxPool 2x2	53 by 53	128
Block 3, 3x Conv 3x3 VALID	47 by 47	256
Pool 3, MaxPool 2x2	23 by 23	256
Block 4, 3x Conv 3x3 VALID	17 by 17	512

Weights are initialised from ImageNet pretrained VGG16, specifically torchvision VGG16_Weights.IMAGENET1K_V1. The kernel weights copy directly from the SAME padded model since the kernels are the same shape, only the spatial trimming differs.

7.1.2 Network A, Hypercolumn to Aerial Semantic Logits

The hypercolumn fuses multi scale features from all four VGG blocks. Each feature map is bilinearly upsampled to the target 17 by 17 resolution and concatenated channel wise.

Source	Resolution	Channels	Notes
f1	220 by 220 to 17 by 17	64	Very fine detail, early edges

Source	Resolution	Channels	Notes
f2	106 by 106 to 17 by 17	128	Local textures and patterns
f3	47 by 47 to 17 by 17	256	Mid level shapes, object parts
f4	17 by 17	512	High level semantics
Concatenation	17 by 17	960	Total hypercolumn

The 960 channel hypercolumn is processed by a three layer 1 by 1 convolution MLP, which is effectively a per pixel fully connected network applied independently at each of the 17 by 17 spatial locations. The structure is 960 channels into 512, ReLU, into 512, ReLU, into 4 (the number of classes). 1 by 1 convolutions are used because at this stage we predict a semantic label per aerial pixel without mixing information across spatial locations. That mixing happens later in the transformation. The output, called L_a , contains raw unnormalised logits for the four semantic classes at each of the 17 by 17 aerial locations.

7.1.3 Network S, Conditioning Scalar

The conditioning network reads the deep conv4_3 features of shape (B, 512, 17, 17) and produces a per pixel scalar of shape (B, 1, 17, 17). Its job is to encode local aerial content (for example, building presence or vegetation density) so that the transformation matrix can adapt to the actual aerial image rather than producing a generic projection. The structure is Conv 512 to 64 with ReLU, then Conv 64 to 1.

7.1.4 Indexing Tensor (Pre computed, Non trainable)

A fixed buffer of shape (n_src, n_tgt, 4) stores the normalised position coordinates (i, j, y, x) for every pair of source aerial pixel and target ground pixel, with all coordinates in the range zero to one. This gives the transformation network a geometric prior so that it does not need to learn relative spatial relationships from scratch.

7.1.5 Network F, Transformation Weight MLP

For the conditioned variant of the model, the input to Network F is formed by concatenating the indexing tensor with the tiled conditioning scalar S, yielding 293 channels per entry, that is four coordinate dimensions plus 289 conditioning scalars (one for every aerial pixel). The MLP structure is.

- Conv 293 to 128, kernel 1, ReLU.
- Conv 128 to 64, kernel 1, ReLU.
- Conv 64 to 1, kernel 1.

Output is raw weight logits of shape (B, 1, 289, 320), squeezed to (B, 289, 320). A softmax is then applied over the source (aerial) dimension so that the weights summed over all source pixels equal one for each

target ground pixel. The result M is the transformation matrix. For each ground pixel t , M is a probability distribution over the 289 aerial pixels that says which aerial locations contribute to the semantics of that ground location.

7.1.6 Transfer Operation

The ground logits are computed as L_g equals L_a (flattened) batch matmul M plus ground bias, where the batch matrix multiplication maps $(B, C, 289)$ by $(B, 289, 320)$ to $(B, C, 320)$. The output is reshaped back to $(B, C, 8, 40)$ and a learnable ground bias of shape $(1, C, 8, 1)$ is added. The bias is one value per ground row, broadcast across all columns and all batches. It captures the systematic prior that certain ground rows tend to correspond to certain classes. The top rows near the horizon are usually building or sky, the bottom rows are usually road. Letting the network store this prior in the bias frees the transformation matrix to focus on the harder image dependent part.

7.1.7 CrossNet Parameter Count

Module	Approximate Parameters
VGG16 backbone (4 blocks only, no fully connected)	about 7.6 M
Aerial MLP (Network A)	about 1.1 M
Conditioning network (Network S)	about 33 K
Weight network (Network F), 293 to 128 to 64 to 1	about 54 K
Ground bias	32 parameters
Total trainable	about 8.8 M

7.2 Stage 2, Pix2Pix U Net GAN

The second stage takes a 4 channel one hot semantic map at 256 by 512 resolution, normalised to the range minus one to plus one, and produces a 3 channel RGB image at the same resolution. We use a U Net generator with eight encoder and decoder stages and a 70 by 70 PatchGAN discriminator. A standard CNN regressed against L1 or L2 pixel loss would produce blurry outputs because averaging minimises L2, and the average of many sharp plausible photos is a blurry photo. The adversarial loss in the GAN forces high frequency, sharp detail.

7.2.1 Generator, 8 Stage U Net

The encoder uses a DownBlock pattern of Conv2d (4 by 4, stride 2, padding 1), then InstanceNorm, then LeakyReLU(0.2), except the first encoder block which has no normalisation.

Stage	Output Size	Channels	Notes
e1	128 by 256	64	No InstanceNorm
e2	64 by 128	128	
e3	32 by 64	256	
e4	16 by 32	512	
e5	8 by 16	512	
e6	4 by 8	512	
e7	2 by 4	512	
bottleneck	1 by 2	512	Conv only, no norm
d1 plus skip e7	2 by 4	512	Dropout 0.5
d2 plus skip e6	4 by 8	512	Dropout 0.5
d3 plus skip e5	8 by 16	512	Dropout 0.5
d4 plus skip e4	16 by 32	512	
d5 plus skip e3	32 by 64	256	
d6 plus skip e2	64 by 128	128	
d7 plus skip e1	128 by 256	64	
out	256 by 512	3	ConvTranspose with Tanh

Skip connections concatenate the encoder feature maps to the corresponding decoder level, doubling the input channel count at each decoder block. Dropout with $p=0.5$ on the first three decoder levels acts as stochastic regularisation. The final Tanh maps the output to the range minus one to plus one.

Instance Normalisation is used in place of Batch Normalisation because we train with very small batches (batch size two on a 4 GB GPU). Batch norm computes statistics over the entire batch and is very noisy at batch size two. Instance norm normalises each image independently using its own mean and variance, which is stable regardless of batch size.

7.2.2 Discriminator, 70 by 70 PatchGAN

The discriminator receives the concatenation of the semantic label and the RGB image, with input shape (B, 4 plus 3, 256, 512) equal to (B, 7, 256, 512). It is a conditional discriminator. It does not just judge

whether the image looks real, it judges whether the image is a plausible realisation of the given semantic layout. This stops the generator from producing a sharp but completely wrong scene.

The architecture is a stack of strided 4 by 4 convolutions.

- Conv 7 to 64, $k=4$, $s=2$, $p=1$, LeakyReLU(0.2), no norm.
- Conv 64 to 128, $k=4$, $s=2$, $p=1$, InstanceNorm, LeakyReLU(0.2).
- Conv 128 to 256, $k=4$, $s=2$, $p=1$, InstanceNorm, LeakyReLU(0.2).
- Conv 256 to 512, $k=4$, $s=1$, $p=1$, InstanceNorm, LeakyReLU(0.2).
- Conv 512 to 1, $k=4$, $s=1$, $p=1$, no activation.

The output is a patchwise real or fake score map of size approximately $(B, 1, 30, 62)$. There is no sigmoid because we use raw logits with the LSGAN MSE loss. The roughly 70 by 70 receptive field means the discriminator judges local patches rather than the whole image. This focuses adversarial training on local texture realism, which is exactly what makes the visual difference between a photorealistic and a blurry output.

8. Loss Functions

8.1 CrossNet Loss, Soft Cross Entropy

The ground semantic logits L_g of shape $(B, C, 8, 40)$ are compared against the soft target produced by one hot encoding the full resolution label and bilinearly downsampling it to the logit size $(8, 40)$. The loss is the standard soft cross entropy.

In code this is `loss equals minus (soft_lbl times F.log_softmax(logits, dim=1)).sum(dim=1).mean()`. Only the ground logits L_g appear in the loss. The aerial logits L_a are not directly supervised. They are pulled into shape implicitly through the transfer $L_g \text{ equals } L_a \text{ matmul } M \text{ plus bias}$. If the aerial prediction is wrong, the ground prediction will be wrong, and the gradient flows back through M into L_a . This is what makes the framework weakly supervised aerial semantic labelling.

Gradient clipping with `max_norm` equal to 5.0 is applied after the AMP scaler unscales the gradients, which prevents the large weight matrix computation from producing exploding gradients in early training when the conditioning network has not yet learned anything useful.

8.2 GAN Losses, LSGAN plus L1

The GAN uses Least Squares GAN (LSGAN) objectives, which use MSE in place of binary cross entropy.

The motivation is gradient stability. Standard binary cross entropy GAN loss vanishes when the discriminator becomes confident, leaving no signal for the generator. MSE always provides a meaningful gradient as long as the predicted score is not at the target.

The loss term breakdown is below.

Term	Formula (informal)	Weight
Discriminator real loss	$\text{MSE}(D_{\text{real}}, 0.9)$	x 0.5
Discriminator fake loss	$\text{MSE}(D_{\text{fake}}, 0)$	x 0.5
Generator adversarial	$\text{MSE}(D(G(\text{label})), 1)$	x 1
Generator reconstruction (L1)	absolute difference between $G(\text{label})$ and x_{real}	x 100

The real target is 0.9 instead of 1.0, which is one sided label smoothing. It stops the discriminator from saturating and producing brittle gradients. The factor of 0.5 on the discriminator total slows the discriminator relative to the generator. The very high weight of 100 on the L1 reconstruction term is taken from the original Pix2Pix paper. It ensures the generator produces structurally correct images, while the adversarial loss adds the high frequency sharpness that makes the result look photorealistic. We use L1 instead of L2 because L1 produces sharper outputs. It does not penalise large boundary errors quadratically, so it does not encourage the generator to hedge by blurring.

9. Hyperparameters

9.1 CrossNet Hyperparameters

Parameter	Value	Rationale
Learning rate	1e minus 3 (initial)	Standard Adam starting LR for VGG scale networks
LR decay	x 0.7 every 5,000 steps (staircase exponential)	Mirrors the original TF1 implementation, exponential_decay
Batch size	4	Limited by 4 GB VRAM (Quadro P1000), AMP enables FP16 forward pass
Epochs	10	About 8,883 steps per epoch times 10 equals 88,830 total steps
Optimizer	Adam (default betas, beta1 equal 0.9, beta2 equal 0.999)	Adaptive learning rate, standard for cross entropy classification
Gradient clipping	max_norm equal 5.0	Prevents early instability with the large weight matrix computation
AMP	Enabled (FP16 forward, FP32 master weights)	Reduces VRAM, allows batch equal 4 on a 4 GB card

Parameter	Value	Rationale
num_classes	4	Sky, vegetation, building, road (CVUSA annotation scheme)
source_size	(17, 17) equals 289 aerial pixels	Determined by VALID padded VGG16 for 224 by 224 input
target_size	(8, 40) equals 320 ground pixels	Paper defined, the ratio reflects the panorama aspect ratio
num_workers	0	Windows compatibility (multiprocessing limitations)
save_every	500 steps	About 17 minutes between saves at the estimated step rate

Learning rate schedule at key steps. Steps 0 to 5,000 use lr equal to $1e^{-3}$. Steps 5,001 to 10,000 use lr equal to $7e^{-4}$. Steps 10,001 to 15,000 use lr equal to $4.9e^{-4}$. Steps 15,001 to 20,000 use lr equal to $3.43e^{-4}$, and so on with $\times 0.7$ every 5,000 steps. By step 88,830, after 17 decay events, the learning rate has dropped to roughly $1.4e^{-5}$.

Selection methodology. The learning rate and decay schedule are inherited directly from the original CrossNet paper, specifically `TF1 exponential_decay(0.001, global_step, 5000, 0.7)`. Batch size and AMP settings were determined empirically to fit on the available 4 GB GPU.

9.2 GAN Hyperparameters

Parameter	Value	Rationale
Learning rate	$2e^{-4}$	Standard Pix2Pix and DCGAN learning rate (Isola et al. 2017)
LR schedule	Linear decay to zero over the second half of training	Standard Pix2Pix schedule, maintains sharpness then reduces noise
Batch size	2	About 2.3 GB VRAM on the P1000 with AMP, fits safely in 4 GB
Epochs	10 plus	Training extended, checkpoint at step 197,766 (about 11 epochs at batch equal 2)
Optimizer (G and D)	Adam, beta1 equal 0.5, beta2 equal 0.999	beta1 equal 0.5 is standard for GAN training, faster momentum reset
lambda L1	100	From the Pix2Pix paper, pixel loss dominates adversarial for structural fidelity

Parameter	Value	Rationale
ngf and ndf	64 and 64	Standard Pix2Pix filter counts, balanced quality versus VRAM
G steps per D step	2	Prevents discriminator domination, keeps adversarial gradient alive
Label smoothing	0.9 on real targets	One sided smoothing, prevents D overconfidence on real samples
Training resolution	256 by 512	Power of two dimensions for clean U Net, upsampled to 224 by 1232 at inference
AMP	Enabled	Same as CrossNet
save_every	2,000 steps	Reduces I/O overhead for the longer running GAN

10. Critical Analysis

10.1 Limitations Ordered by Severity

Our analysis identified ten limitations in the original paper, ordered by severity and practical impact.

#	Limitation	Impact
1	Noisy weak supervision. SegNet labels are dated and were trained on urban scenes only.	High
2	Only four semantic classes, which is insufficient for real world deployment.	High
3	A linear transformation matrix cannot model complex projective distortions.	High
4	Outdated backbone (VGG16, 138 M parameters), no residual connections, no modern efficiency.	High
5	No spatial attention. Network F conditions globally and ignores spatial relevance.	Medium
6	Low spatial resolution (17 by 17 to 8 by 40) limits fine grained segmentation.	Medium
7	Strict north alignment assumption limits deployment without GPS metadata.	Medium
8	Temporal mismatch between aerial and ground captures. Transient objects cause label noise.	Medium
9	No uncertainty modelling. Predictions are overconfident in occluded regions.	Low
10	Manual preprocessing per dataset limits scalability.	Low

10.2 Research Gaps and Opportunities

From the critiques above, we identified eight concrete research gaps with varying feasibility and impact.

Gap	Description	Feasibility	Impact
Richer Supervision	Replace SegNet with DeepLabV3+ or SegFormer	2 to 3 days	High
Modern Backbone	Use ResNet50 or MobileNetV2	1 to 2 days	High
Attention Mapping	Add spatial or cross attention to transformation F	2 to 3 days	High
Multi scale Features	Add conv5 plus FPN fusion	1 day	Medium
Data Augmentation	Flip, jitter, rotation	0.5 day	Medium
Temporal Robustness	Filter pairs by seasonal similarity	2 to 3 days	Medium
More Classes (6 plus)	Match the ISPRS taxonomy	1 day	Medium
Photorealistic Synthesis	Add a Pix2Pix GAN on top of the semantic pipeline	Done in this project	High

11. Codebase Modernisation

The original open source implementation at github.com/viiibridges/crossnet targets Python 2 and TensorFlow 1.1, both of which are end of life. Our team carried out a full PyTorch 2.x rewrite. The legacy TensorFlow code remains in the repository under filenames suffixed with `_tf_legacy.py` for reference, but it is not used for actual training. Below is a summary of the compatibility and migration issues that had to be resolved.

File	Issue	Fix Applied
main.py	Private <code>__flags</code> attribute access on <code>tf.flags.FLAGS</code>	Replaced with <code>vars(FLAGS)</code>
crossnet.py	<code>xrange</code> removed in Python 3	Replaced all instances with <code>range</code>
crossnet.py	Deprecated loss API signature in TF2	Added <code>logits</code> and <code>labels</code> keyword arguments
crossnet.py	Softmax axis unspecified	Added axis equal to minus one explicitly
misc.py	<code>scipy.misc.imread</code> removed	Replaced with <code>PIL.Image</code> and <code>skimage.io</code>

File	Issue	Fix Applied
misc.py	xrange calls	Replaced with range
models.py	dim argument removed in TF2	Replaced with axis
nets.py	contextlib.nested deprecated	Replaced with contextlib.ExitStack
all	TF1 graph and session pattern	Rewritten as PyTorch nn.Module classes with eager execution
all	TF1 exponential_decay learning rate	Reproduced as a manual staircase schedule in PyTorch

Beyond direct API replacements, the entire training graph was rewritten. The TF1 legacy code uses `tf.slim` regularisation losses which the PyTorch version omits. This is a minor difference that may very slightly favour the PyTorch implementation, since the absence of the small L2 regularisation term lets the weights move slightly more freely.

12. Proposed Improvements

Three improvements were proposed and prototyped during the project. They were not all integrated into the final trained model in order to keep the baseline reproducible, but each one targets a specific limitation identified in Section 9.1.

12.1 Backbone Upgrade, VGG16 to ResNet50

We propose replacing the VGG16 feature extractor with a pretrained ResNet50. ResNet50 has residual skip connections that mitigate vanishing gradients and uses about 82 percent fewer parameters (25 M against 138 M for full VGG16), making it more suitable for aerial imagery where fine structural detail and parameter efficiency both matter.

- Update the hypercolumn so that it draws from the residual stages res2, res3, res4 and res5.
- Add an FPN style multi scale fusion to aggregate features at different receptive fields.
- Expected gain. Roughly 6 percentage points in mean IoU on CVUSA.

12.2 CBAM Spatial Attention in the Transformation Network

We propose adding Convolutional Block Attention Modules (CBAM) inside the transformation network F. CBAM applies channel attention followed by spatial attention, letting the network focus on semantically relevant regions of the aerial feature map when predicting each element of the transformation matrix M .

- Channel attention recalibrates inter channel dependencies through global average and max pooling.

- Spatial attention highlights salient locations using a 7 by 7 convolution over pooled channel descriptors.
- Expected gain. Improved transformation precision at region boundaries such as roads and building edges.

12.3 DeepLabV3+ Supervision

SegNet, used in the original paper to generate pseudo labels, was trained primarily on the urban CamVid dataset and is architecturally limited for modern use. We propose replacing it with DeepLabV3+ based on a ResNet101 backbone trained on Cityscapes plus ADE20K, which provides cleaner and more complete segmentation masks as training targets.

- Richer class vocabulary, which enables future extension beyond four classes.
- Better handling of suburban and rural scenes that dominate CVUSA.

12.4 Architecture Comparison, Original versus Improved

The following table summarises the architectural differences across all components of the pipeline.

Component	Original (Zhai et al. 2017)	Improved (Proposed)
Backbone	VGG16, 138 M parameters, no skip connections	ResNet50, 25 M parameters, residual connections
Feature Extraction	Hypercolumn from conv1 to conv4	FPN style fusion of res2, res3, res4, res5
Transformation	Linear MLP, no spatial awareness	CBAM attention (channel and spatial)
Supervision	SegNet pseudo labels, urban biased	DeepLabV3+ pseudo labels, broader scenes
Augmentation	None	Horizontal flip, colour jitter
Framework	TensorFlow 1.1, Python 2	PyTorch 2.x, Python 3
Classes	Four (road, vegetation, building, sky)	Four, extendable to six via ISPRS taxonomy

12.5 Pix2Pix GAN Extension, Photorealistic Panorama Synthesis

Beyond semantic segmentation, we extended the inference pipeline to produce a photorealistic ground level panorama. This is the one improvement that was fully implemented and trained, and it is described in detail in Section 7.2. The two stage pipeline goes aerial 224 by 224, then CrossNet, then semantic logits 8 by 40, then argmax, then one hot, then upsample to 256 by 512, then Pix2Pix GAN, then RGB at 224 by 1232 (after final bilinear upsampling for display).

The GAN follows a Pix2Pix LSGAN design.

Component	Architecture	Parameters
Generator	8 stage U Net, ngf equal to 64	about 54 M
Discriminator	70 by 70 PatchGAN, ndf equal to 64	about 2.8 M
Objective	LSGAN (MSE) plus L1 reconstruction (lambda equal to 100)	n/a
Training resolution	256 by 512 with AMP, fits below 2 GB VRAM	n/a

This design was chosen over CycleGAN (paired data is available so cycle consistency is unnecessary), Pix2PixHD (too heavy for a 4 GB GPU) and SPADE or GauGAN (disproportionate complexity for this scale).

13. Implementation Plan

13.1 Week 1, Baseline Setup

- Port the entire codebase from Python 2 to Python 3.
- Migrate TensorFlow 1.1 graph mode operations to PyTorch 2.x equivalents.
- Run a forward pass on sample CVUSA data.
- Verify the pipeline shape sequence, conv1 to conv4, then 17 by 17 hypercolumn, then transformation matrix, then 8 by 40 output.
- Configure TensorBoard logging for histograms and scalars.

13.2 Week 2, Core Improvements

- Replace VGG16 with ImageNet pretrained ResNet50 and update the hypercolumn extraction.
- Implement the CBAM attention module and integrate it into the transformation network F.
- Replace SegNet pseudo label generation with DeepLabV3+ precomputed labels.
- Add data augmentation, horizontal flip and colour jitter.

13.3 Week 3, Evaluation and GAN Extension

- Train baseline (VGG16) and improved (ResNet50 plus Attention) models on CVUSA.
- Evaluate per class IoU, mean IoU, orientation mean angular error.
- Train Pix2Pix GAN on (semantic map, panorama) pairs (5,000 samples, 20 epochs for the demo).
- Generate visualisations laid out as aerial, semantic prediction, real ground, synthesised GAN output.

14. Training Progress

The inference scripts hard code specific checkpoint paths that reveal exactly how far each model was trained.

14.1 CrossNet Checkpoint

- Checkpoint path. outputs/ckpts_pt/crossnet_step0088830.pt.
- Steps. 88,830.
- Steps per epoch. 35,532 divided by 4 (batch) equals 8,883 steps per epoch.
- Epochs completed. 88,830 divided by 8,883 equals exactly 10.0 epochs, a complete training run.

14.2 GAN Checkpoint

- Checkpoint path. outputs/ckpts_gan/G_step0197766.pt.
- Steps. 197,766.
- Steps per epoch. 35,532 divided by 2 (batch) equals 17,766 steps per epoch.
- Epochs completed. 197,766 divided by 17,766 equals about 11.1 epochs, slightly more than ten epochs (likely a resume plus an extra epoch).

Both models were trained to completion on an NVIDIA Quadro P1000 with 4 GB VRAM under CUDA 12.x, using mixed precision (AMP) throughout. The CrossNet docstring explicitly calls out this hardware.

15. Training Logs and Empirical Results

TensorBoard training logs were captured on 26 April 2026 on the host DESKTOP-2QHVFM9 across two active training sessions. The logs record weight histograms for two sub networks, condition_net (the aerial scene descriptor S) and weight_net (the transformation matrix predictor F).

15.1 Training Sessions Timeline

Session	Start Time	Steps Logged	Purpose
Session 1	11:07:17	Step 1 only	Pipeline sanity check, single forward pass to verify graph initialisation
Session 2	11:28:21	Steps 1, 51, 101	Main training run, 101 steps with histogram snapshots every 50 steps
Sessions 3 to 5	11:51 to 11:53	None (graph only)	Short probe runs, likely testing checkpoint loading or inference mode

15.2 Weight Distribution Analysis

Histograms were logged for five layer activations across the two transformation sub networks. The table below summarises the distribution statistics (mean and standard deviation) at each checkpoint.

Layer	Network	Step 1, Mean and Std	Step 51, Mean and Std	Step 101, Mean and Std
condition_net/fc1	S, scene descriptor	0.0262, 0.0342	0.0023, 0.0037	0.0029, 0.0046
condition_net/fc2	S, scene descriptor	0.0391, 0.0144	0.0131, 0.0001	0.0215, 0.000002
weight_net/fc1	F, transformation	0.0344, 0.0522	0.0331, 0.0505	0.0415, 0.0585
weight_net/fc2	F, transformation	0.0296, 0.0444	0.0348, 0.0475	0.0472, 0.0620
weight_net/fc3	F, transformation output	plus 0.0662, 0.0200	minus 0.0847, 0.0573	minus 0.1789, 0.1331

15.3 Interpretation

condition_net (S, Scene Descriptor)

The condition_net shows a clear collapse in activation variance by step 51. The standard deviation of fc2 drops from 0.014 at step 1, to 0.000055 at step 51, to 0.000002 at step 101. A near zero spread of this kind indicates that the scene descriptor is converging to an almost constant output regardless of input. This is consistent with a known failure mode in the original CrossViewNet architecture. When the transformation network F dominates, the conditioning signal from S is effectively ignored. This directly validates our critique that the original linear transformation cannot adequately adapt to aerial image content.

weight_net (F, Transformation Predictor)

The weight_net shows healthy, progressive learning dynamics. The output layer fc3 undergoes a sign reversal between step 1 (mean equal to plus 0.088) and step 51 (mean equal to minus 0.085), with the magnitude and spread continuing to grow through step 101 (mean equal to minus 0.179, std equal to 0.133). This shows the network is actively adjusting the transformation matrix M to minimise cross entropy loss. The inner layers fc1 and fc2 maintain stable ReLU positive distributions throughout, suggesting no gradient vanishing in the feature layers.

Overall Training Health

This logged run only reached 101 steps, a very early snapshot. No loss curves were written to the logs, only histograms, which suggests the logging configuration was set to histogram only mode for this session. The three empty probe sessions afterwards indicate the team was testing inference and deployment mode after the initial training. The full 88,830 step run that produced the final checkpoint was performed in a separate session that was not configured for histogram logging. Key observations from the available logs.

- `weight_net` is learning. Transformation weights are actively diverging in the correct direction.
- `condition_net` is collapsing. Aerial image conditioning degrades by step 51, confirming the S network limitation identified in our critique.
- No signs of gradient explosion in `fc1` or `fc2`. Std stays stable at roughly 0.05.
- Training date confirmed as 26 April 2026, consistent with the research plan metadata.

15.4 Implications for Our Improvements

The `condition_net` collapse observed in the logs directly motivates our CBAM attention improvement (Section 11.2). Replacing the global MLP based conditioning with spatially aware channel and spatial attention should ensure that aerial image content meaningfully modulates the transformation at every spatial location, preventing the collapse to a near constant descriptor. The `weight_net` `fc3` sign reversal also suggests that with more training steps and a stronger ResNet50 backbone, the transformation matrix `M` will reach a more stable and more expressive equilibrium.

15.5 Healthy Loss Ranges

Based on the training script guidance embedded in the code, the expected healthy loss ranges are as follows. CrossNet loss should decrease from roughly 1.0 to 1.4 in early training to roughly 0.3 to 0.6 after ten epochs, which is typical for four class softmax cross entropy. For the GAN, the discriminator loss `D_loss` should sit in the range 0.05 to 0.50 and the generator adversarial term `G_adv` should sit in the range 0.5 to 1.5. These ranges match the empirical training behaviour observed during our run.

16. State of the Art Comparison (SOTA)

The baseline for this project is the original CrossNet paper (Zhai et al., CVPR 2017), which introduced this task and this architecture. That paper reports approximately mIoU equal to 0.42 to 0.46 on four class ground view segmentation of CVUSA, depending on the variant (conditioned versus unconditioned). For a four class problem, random prediction would give an mIoU of roughly 0.25, and a trivial always predict

the most common class baseline might reach 0.30. So 0.42 represents meaningful learning, but leaves significant room for improvement.

Our PyTorch implementation reproduces the original architecture, the same hyperparameters (lr, decay, batch size), the same loss function (soft cross entropy with bilinear downsampled labels), and the same dataset. We expect to reproduce approximately the same mIoU of about 0.42 on the segmentation task.

16.1 Estimated Performance Across Configurations

Configuration	Classes	Estimated mIoU	Status
VGG16 plus SegNet (Original Paper, CVPR 2017)	4	about 0.42	Published baseline
Our Implementation (VGG16 plus SegNet, PyTorch)	4	about 0.42	Reproduced baseline
VGG16 plus DeepLabV3+ supervision	4	about 0.50 plus	Proposed improvement A
ResNet50 plus SegNet supervision	4	about 0.48 plus	Proposed improvement B
ResNet50 plus CBAM plus DeepLabV3+	4	about 0.55 plus	Primary target
ResNet50 plus FPN plus 6 class DeepLab	6	TBD	Stretch goal

The combined improvement of a stronger backbone, attention guided transformation, and cleaner supervision is expected to yield roughly a 13 point gain in mIoU over the original baseline. Individual contributions can be ablated by disabling each improvement independently.

16.2 Qualitative Comparison Against Other Methods

Method	Year	Backbone	Supervision	Output
CrossNet original (Zhai et al.)	CVPR 2017	VGG16	SegNet pseudo labels	Semantic map
Ours (CrossNet PyTorch plus Pix2Pix)	2026	VGG16 (pretrained)	SegNet pseudo labels	Semantic map plus RGB synthesis
Shi et al.	CVPR 2020	ResNet50	Full annotation	Semantic map
TransGeo (Zhu et al.)	CVPR 2022	ViT	Full annotation	Retrieval and segmentation

16.3 Unique Contribution

While the CrossNet backbone follows the original paper, our system adds a complete Pix2Pix GAN second stage that synthesises photorealistic RGB ground panoramas from the predicted semantic maps. This enables an end to end aerial to RGB ground synthesis pipeline, a capability not present in the original CrossNet paper. The pipeline runs as aerial photo at 224 by 224, then CrossNet (VGG16 plus hypercolumn plus conditioned M) producing a semantic map at 8 by 40 and upsampled to 224 by 1232, then Pix2Pix U Net GAN at 256 by 512 producing a synthesised RGB panorama upsampled to 224 by 1232. The full inference runs in under one second on the Quadro P1000.

16.4 Honest Appraisal of Limitations

It is important to be honest about what this work does and does not achieve.

- The supervision is weak. Labels from SegNet are not ground truth, they are the output of another imperfect model. The network is learning to predict predictions. This explains a significant portion of the mIoU gap to methods that use human annotated labels.
- Only four semantic classes. The four class setup is a simplification. Real world deployment would need at least 10 to 20 classes to be useful.
- Low spatial resolution. The CrossNet output is 8 by 40 equals 320 pixels for the entire panorama. Even when bilinearly upsampled to 224 by 1232 for display, this is extremely coarse.
- The linear transformation M. The assumption that aerial to ground mapping can be approximated as a weighted sum of aerial pixels is a strong simplification. Complex perspective effects, occlusion and viewpoint dependent appearance changes are not captured by a linear transform.
- GAN synthesis quality. The GAN produces synthesised panoramas at 256 by 512 training resolution, upsampled to 224 by 1232. The quality is limited by the semantic map quality, by the training resolution, and by the inherent difficulty of going from a coarse 8 by 40 semantic map to a 256 by 512 photo.

17. Inference Pipeline

At inference time the full pipeline operates as follows.

- A 224 by 224 aerial image is centre cropped, normalised to the range minus one to plus one, and fed into the CrossNet model.
- CrossNet's backbone extracts a 17 by 17 feature pyramid, computes the 960 channel hypercolumn, predicts aerial semantic logits L_a at 17 by 17, computes the conditioning scalar S , constructs the 289 by 320 transformation matrix M , and transfers L_a through M to obtain ground logits L_g at 8 by 40.

- Lg is argmax decoded to an 8 by 40 class map, one hot encoded, and upsampled to 256 by 512 using nearest neighbour interpolation, which preserves sharp class boundaries.
- The 256 by 512 one hot map (normalised to the range minus one to plus one) is fed through the U Net Generator, which synthesises a 256 by 512 RGB image through eight upsampling decoder stages with skip connections.
- The synthesised image is bilinearly upsampled to 224 by 1232 for display.
- A visual montage is saved showing the aerial input, the CrossNet semantic map and the GAN synthesised panorama side by side.

The entire inference pipeline runs in under one second on the Quadro P1000 GPU.

18. File by File Role Summary

The following table summarises the role of each significant file in the project repository.

File	What It Does
crossnet_model.py	Defines the full CrossNet model, backbone, hypercolumn, Network A, Network S, Network F, transfer.
crossnet_dataset.py	PyTorch Dataset, loads CSV, preprocesses aerial, ground and label for CrossNet training.
train_crossnet.py	Training loop, Adam, soft cross entropy, AMP, gradient clipping, checkpointing and visualisation.
gan_unet_model.py	U Net Generator and PatchGAN Discriminator architecture definitions.
train_gan_pix2pix.py	GAN training loop, LSGAN loss, separate G and D optimisers, linear LR decay, AMP.
aerial_to_ground_final.py	End to end inference, loads both models, preprocesses input, runs both stages, saves the montage.
infer_random_demo.py	Demo, picks a random training sample and runs the full pipeline, saves comparison outputs.
prepare_data.py	Utility to generate cvpr_train.csv and cvpr_val.csv from the raw CVUSA directory structure.
config_tf_legacy.py	TF1 flags, original hyperparameters (batch equal 8, lr equal 0.001, epochs equal 10), legacy reference.
crossnet_tf_legacy.py	Original TF1 CrossNet graph construction, loss and Adam optimiser.
train_crossnet_tf_legacy.py	TF1 training loop (legacy, not used for actual training).
backbone_tf_legacy.py	TF slim VGG like backbone (legacy).

File	What It Does
layers_tf_legacy.py	TF1 hypercolumn, indexing and transfer functions (legacy).
aerial_to_ground_final.ipynb	Jupyter notebook interface to the inference pipeline with a file picker.
cvpr_train.csv, cvpr_val.csv	Data manifests, 35,532 and 8,884 lines of aerial, ground and label paths.
docs/research_plan.md	Research analysis, critique of the original paper, proposed improvements and expected metrics.

19. Conclusion

CrossViewNet demonstrates that meaningful aerial image semantics can be learned without manual annotation, by leveraging co located ground imagery as a free source of supervision. Our reproduction confirmed the core claims of the original paper and exposed both architectural and codebase limitations that constrain its practical utility.

Our work makes three concrete contributions. First, a complete PyTorch reimplementation of the CrossNet paper, replacing the unsupported TensorFlow 1.1 codebase, with mixed precision support that lets the model train on a 4 GB consumer GPU. Second, a working Pix2Pix conditional GAN that converts the predicted semantic maps into photorealistic RGB ground panoramas, extending the original system from semantic prediction to full scene synthesis. Third, a detailed critique with a three pronged improvement strategy (ResNet50 backbone, CBAM spatial attention, DeepLabV3+ supervision) that addresses the most impactful limitations of the original design and is set up for future ablation work.

Both models trained to completion on a single 4 GB Quadro P1000, with CrossNet finishing a full ten epoch training run at 88,830 steps and the GAN finishing about 11 epochs at 197,766 steps. The end to end aerial to ground synthesis pipeline runs in under one second per query, which makes it practical for interactive use on a laptop. The combined improvement target of roughly a 13 point gain in mean IoU over the original 0.42 baseline is the next milestone for follow up work.

20. References

[1] Original Research Paper (Primary).

Zhai, M., Bessinger, Z., Workman, S., and Jacobs, N. (2017). Predicting Ground Level Scene Layout from Aerial Imagery. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017), pp. 867 to 875. arXiv:1612.02709.

[2] Related Work, Cross View Geo Localization.

Shi, Y., Liu, L., Yu, X., and Li, H. (2019). Spatial Aware Feature Aggregation for Cross View Image Based Geo Localization. Advances in Neural Information Processing Systems (NeurIPS 2019). This paper proposes a spatial aware feature aggregation network (SAFA) that learns structured correspondences between satellite and ground level image pairs for geo localization, directly extending the cross view learning paradigm introduced by CrossViewNet. The method significantly improves retrieval accuracy on CVUSA and CVACT datasets.

[3] Related Work, Satellite to Street Image Synthesis.

Regmi, K., and Borji, A. (2018). Cross View Image Synthesis Using Geometry Guided Conditional GANs. In Proceedings of the European Conference on Computer Vision (ECCV 2018), pp. 388 to 404. This paper directly addresses the cross view image synthesis problem, generating realistic street level images from overhead satellite or aerial views, using conditional GAN frameworks with geometric priors. It shares the aerial to ground generation objective with our Pix2Pix extension and provides a strong methodological baseline for evaluating synthesised panorama quality.

[4] Backbone, Deep Residual Learning.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016), pp. 770 to 778. Introduces the ResNet architecture adopted in our improved backbone.

[5] Attention, CBAM.

Woo, S., Park, J., Lee, J. Y., and Kweon, I. S. (2018). CBAM, Convolutional Block Attention Module. In Proceedings of the European Conference on Computer Vision (ECCV 2018), pp. 3 to 19. Introduces the channel and spatial attention module integrated into our transformation network.

[6] Supervision, DeepLabV3+.

Chen, L. C., Zhu, Y., Papandreou, G., Schroff, F., and Adam, H. (2018). Encoder Decoder with Atrous Separable Convolution for Semantic Image Segmentation. In Proceedings of the European Conference on Computer Vision (ECCV 2018), pp. 801 to 818. Introduces the DeepLabV3+ model used as our improved pseudo label generator.

[7] GAN, Pix2Pix.

Isola, P., Zhu, J. Y., Zhou, T., and Efros, A. A. (2017). Image to Image Translation with Conditional Adversarial Networks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017), pp. 1125 to 1134. Foundation for the Pix2Pix GAN stage of our panorama synthesis pipeline.

[8] Hypercolumns.

Hariharan, B., Arbelaez, P., Girshick, R., and Malik, J. (2015). Hypercolumns for Object Segmentation and Fine Grained Localization. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2015). Introduces the multi scale per pixel feature representation used in Network A.

[9] LSGAN.

Mao, X., Li, Q., Xie, H., Lau, R. Y. K., Wang, Z., and Smolley, S. P. (2017). Least Squares Generative Adversarial Networks. In Proceedings of the IEEE International Conference on Computer Vision (ICCV 2017), pp. 2794 to 2802. Provides the MSE based GAN objective used in our generator and discriminator training.

21. Links

GitHub: <https://github.com/Hamdan-Vohra/Aerial2Ground.git> (Branch: pytorch-pipeline)

Google Drive:

https://drive.google.com/drive/folders/1DTnBfnZLHw4TqrbUqmDRHmwFIJBNF1mx?usp=drive_link

(the dataset[cvpr_subset], and output models, logs and dumps)